

Défi facultatif

Ce défi est **facultatif et non noté**. Votre priorité est le **projet Fondation**. Lancez-vous dans ce défi une fois le TP1 terminé.

Ce défi combine les deux cours du 9 octobre. Il mobilise les **objets** puis les **fichiers** et le **JSON**.

Règles du jeu

- Vous codez **vous-mêmes, sans IA** : ni ChatGPT, ni Claude, ni autocomplétion Copilot.
- La documentation Python est autorisée et même encouragée.
- Écrivez votre code dans le fichier `challenge4.py`.
- **Validation** : levez la main, exécutez votre code devant moi puis répondez à une question sur votre code.
- **Notation** : aucune, ce défi est facultatif. Faites-le valider quand même. C'est un bon entraînement pour l'examen.

4

Alertes en objets

Héritage · polymorphisme · fichiers · JSON

SOC

Votre détecteur de brute force du défi 2 fonctionne. Il faut maintenant l'industrialiser comme dans un SIEM : lire un vrai fichier de journal, produire des **alertes typées** et les exporter en JSON pour les autres outils du SOC.

Votre mission

1. Écrivez le journal `AUTH_LOG` du défi 2 (rappelé ci-dessous) dans un fichier `auth.log`, une ligne par élément.
2. Créez une classe `Alert` :
 - un **attribut de classe** `severity = "INFO"` et un attribut `ip` ;
 - une méthode `describe()` qui renvoie `[INFO] activité suspecte depuis <ip>` ;
 - une méthode `to_dict()` qui renvoie `{"severity": ..., "ip": ..., "message": ...}`.
3. Créez deux classes filles qui redéfinissent `severity` et `describe()` :
 - `BruteForceAlert(ip, failures)`, de sévérité `"HIGH"` ;
 - `CompromiseAlert(ip, user)`, de sévérité `"CRITICAL"` : une IP qui réussit à se connecter après un brute force.
4. Écrivez la fonction `analyze(path, threshold=5)`, qui **lit le fichier** et renvoie la liste des alertes.
5. Affichez chaque alerte puis enregistrez-les toutes dans `report.json`.

Données à copier – le journal du défi 2

Données à copier – journal d'authentification

```
AUTH_LOG = [
    "Oct 7 09:12:03 srv-web sshd[2101]: Accepted password for Chris from 192.0.2.10 port 50122 ssh2",
    "Oct 7 09:40:17 srv-web sshd[2144]: Failed password for Manon from 192.0.2.25 port 50388 ssh2",
    "Oct 7 09:40:29 srv-web sshd[2144]: Accepted password for Manon from 192.0.2.25 port 50388 ssh2",
    "Oct 7 10:02:11 srv-web sshd[2210]: Failed password for root from 203.0.113.45 port 51122 ssh2",
    "Oct 7 10:02:14 srv-web sshd[2212]: Failed password for root from 203.0.113.45 port 51130 ssh2",
    "Oct 7 10:02:18 srv-web sshd[2215]: Failed password for invalid user admin from 203.0.113.45 port 51141 ssh2",
    "Oct 7 10:02:21 srv-web sshd[2218]: Failed password for invalid user test from 203.0.113.45 port 51150 ssh2",
    "Oct 7 10:02:25 srv-web sshd[2221]: Failed password for Chris from 203.0.113.45 port 51162 ssh2",
    "Oct 7 10:02:29 srv-web sshd[2224]: Failed password for Chris from 203.0.113.45 port 51170 ssh2",
    "Oct 7 10:02:33 srv-web sshd[2227]: Failed password for Chris from 203.0.113.45 port 51178 ssh2",
    "Oct 7 10:02:36 srv-web sshd[2230]: Accepted password for Chris from 203.0.113.45 port 51185 ssh2",
    "Oct 7 10:02:37 srv-web sshd[2230]: pam_unix(sshd:session): session opened for user Chris by (uid=0)",
    "Oct 7 10:15:40 srv-web sshd[2301]: Failed password for invalid user oracle from 198.51.100.23 port 40022 ssh2",
    "Oct 7 10:15:44 srv-web sshd[2303]: Failed password for invalid user postgres from 198.51.100.23 port 40031 ssh2",
    "Oct 7 10:15:47 srv-web sshd[2305]: Failed password for root from 198.51.100.23 port 40040 ssh2",
    "Oct 7 10:15:51 srv-web sshd[2307]: Failed password for invalid user ubuntu from 198.51.100.23 port 40052 ssh2",
    "Oct 7 10:15:55 srv-web sshd[2309]: Failed password for invalid user pi from 198.51.100.23 port 40060 ssh2",
    "Oct 7 10:16:02 srv-web sshd[2311]: Connection closed by 198.51.100.23 port 40071 [preauth]",
    "Oct 7 11:30:08 srv-web sshd[2402]: Failed password for Cosmo from 198.51.100.77 port 42210 ssh2",
    "Oct 7 11:30:15 srv-web sshd[2404]: Failed password for Cosmo from 198.51.100.77 port 42218 ssh2",
    "Oct 7 11:31:02 srv-web sshd[2410]: Accepted password for Cosmo from 198.51.100.77 port 42230 ssh2",
]
```

Exemples attendus

Résultats attendus

```
alerts = analyze("auth.log")
for alert in alerts:
    print(alert.describe())
# [HIGH] brute force depuis 203.0.113.45 : 7 échecs
# [HIGH] brute force depuis 198.51.100.23 : 5 échecs
# [CRITICAL] 203.0.113.45 s'est connecté au compte Chris après un brute force

print(Alert("192.0.2.99").describe())
# [INFO] activité suspecte depuis 192.0.2.99

print(isinstance(alerts[0], Alert))
# True
```

Indices

- Revoyez les notions d'attribut de classe, d'héritage et de polymorphisme.
- `to_dict()` ne s'écrit **qu'une fois**, dans `Alert`. En appelant `self.describe()`, elle utilisera automatiquement la bonne version.
- Pour que les accents restent lisibles dans `report.json` : `json.dump(..., indent=2, ensure_ascii=False)`.

Pour valider, montrez-moi...

- les trois exemples ci-dessus, qui donnent exactement ces résultats ;
- le fichier `report.json` ouvert dans VS Code.

Bonus

- Triez les alertes de la plus grave à la moins grave.
- Ajoutez une méthode `__str__` pour afficher une alerte avec un simple `print(alert)`.
- Filtrez les alertes par sévérité grâce à un argument de la ligne de commande : `python challenge4.py CRITICAL ( sys.argv )`.